

94. 二叉树的中序遍历

Category	Difficulty	Likes	Dislikes	ContestSlug	ProblemIndex	Score
algorithms	Easy	(Not Available)	(Not Available)	-	-	0

- **Tags:** 栈, 树, 深度优先搜索, 二叉树
- **Companies:** (Not Available)

给定一个二叉树的根节点 `root` , 返回它的 **中序**遍历。

示例 1:

输入 : `root = [1,null,2,3]`
输出 : `[1,3,2]`

示例 2:

输入 : `root = []`
输出 : `[]`

示例 3:

输入 : `root = [1]`
输出 : `[1]`

提示:

- 树中节点数目在范围 `[0, 100]` 内
- `-100 <= Node.val <= 100`

进阶: 递归算法很简单, 你可以通过迭代算法完成吗?

Discussion | Solution

解决方法

1. 问题理解

目标: 对给定的二叉树执行中序遍历, 并返回遍历结果的节点值列表。

中序遍历顺序: 左子树 -> 根节点 -> 右子树。

要求: - 实现递归方法。 - (进阶) 实现迭代方法。

2. 思路分析：递归与迭代

方法一：递归 (Depth-First Search - DFS) 递归是实现树的深度优先遍历（包括前序、中序、后序）最直观、代码最简洁的方式。中序遍历的核心在于访问节点的时机：在处理完左子树之后，处理右子树之前。

1. **函数定义**：定义一个辅助递归函数 `traverse(node)`。
2. **基本情况 (Base Case)**：如果当前节点 `node` 为 `null`（即遍历到空节点），则递归终止，直接返回。这是递归的出口。
3. **递归步骤 (Recursive Step)**：
 - **处理左子树**：调用 `traverse(node.left)`，递归地对当前节点的左子树进行中序遍历。
 - **处理根节点**：当左子树遍历完成后（即 `traverse(node.left)` 返回后），访问当前节点 `node`。通常是将 `node.val` 添加到结果列表 `result` 中。
 - **处理右子树**：调用 `traverse(node.right)`，递归地对当前节点的右子树进行中序遍历。

执行流程示例 (root = [1,null,2,3]): 1. `traverse(1)` 2. `traverse(null)` (1 的左子节点) -> 返回 3. 访问节点 1 -> `result = [1]` 4. `traverse(2)` (1 的右子节点) 5. `traverse(3)` (2 的左子节点) 6. `traverse(null)` (3 的左子节点) -> 返回 7. 访问节点 3 -> `result = [1, 3]` 8. `traverse(null)` (3 的右子节点) -> 返回 9. `traverse(null)` (2 的右子节点) -> 返回 10. 访问节点 2 -> `result = [1, 3, 2]` (这里步骤 5 访问 3 后, `traverse(3)` 返回到 `traverse(2)`, 然后才访问 2, 最后处理 2 的右子树) 更正: 访问节点 2 应该在处理完其左子树 (节点 3) 之后, 处理其右子树之前。* `traverse(2)` 调用 `traverse(3)` * `traverse(3)` 调用 `traverse(null)` (左), 返回 * `traverse(3)` 访问 3 -> `result = [1, 3]` * `traverse(3)` 调用 `traverse(null)` (右), 返回 * `traverse(3)` 结束, 返回到 `traverse(2)` * `traverse(2)` 访问 2 -> `result = [1, 3, 2]` * `traverse(2)` 调用 `traverse(null)` (右), 返回 * `traverse(2)` 结束, 返回到 `traverse(1)` 11. `traverse(1)` 结束。最终 `result = [1, 3, 2]`。

方法二：迭代 (使用栈模拟递归) 迭代方法的核心是使用一个栈 (Stack) 来模拟函数调用栈的行为，从而实现非递归的遍历。

1. **初始化**：
 - 创建一个空栈 `stack`。
 - 创建一个空列表 `result` 用于存储遍历结果。
 - 创建一个指针 `curr`，初始指向根节点 `root`。
2. **循环遍历**：使用 `while` 循环，条件是 `curr` 不为空 或者 `stack` 不为空。这个条件确保了即使根节点为空或者遍历过程中 `curr` 暂时变为空，只要栈中还有待处理的节点，遍历就不会停止。
3. **循环内部逻辑**：
 - **第一阶段：一路向左并将节点入栈**
 - 内层 `while (curr != null)` 循环：只要 `curr` 指向的节点不为空，就执行以下操作：

- * 将当前节点 `curr` 压入 `stack` 中。这相当于递归调用时保存现场（保存当前节点，以便后续回溯访问）。
- * 将 `curr` 移动到其左子节点: `curr = curr.left`。
- 这个内层循环会一直执行，直到找到当前子树的最左下角的节点（此时 `curr` 变为 `null`）。栈中保存了从起始节点到这个最左节点的路径上的所有节点。
- **第二阶段：访问栈顶节点并转向右子树**
 - 当内层 `while` 循环结束时，`curr` 为 `null`。这表示当前路径的左子树已经处理完毕（或者当前节点没有左子树）。
 - 此时，应该访问“根”节点了。这个“根”节点就是栈顶的元素（最近一次入栈的、没有被访问过的最左节点）。
 - 从栈中弹出一个节点: `node = stack.pop()`。
 - 访问该节点: 将 `node.val` 添加到 `result` 列表中。
 - 转向右子树: 将 `curr` 指针指向弹出节点 `node` 的右子节点: `curr = node.right`。这相当于递归中处理完根节点后，开始处理右子树。
 - 之后，外层 `while` 循环继续。如果新的 `curr`（即右子节点）不为空，则重复第一阶段（一路向左）；如果新的 `curr` 为空，则继续第二阶段（弹出栈顶元素访问）。
- 4. **循环结束**: 当外层 `while` 循环的条件 (`curr != null || !stack.isEmpty()`) 不满足时，表示所有节点都已访问完毕，循环结束。
- 5. **返回结果**: 返回 `result` 列表。

迭代过程示例 (`root = [1,null,2,3]`):

步骤	curr	stack	result	操作
1	1	[]	[]	初始化
2	1	[1]	[]	curr 不为空，压栈, <code>curr = curr.left</code> (<code>null</code>)
3	null	[1]	[]	curr 为空，弹栈 (1)，访问 1
4	2	[]	[1]	<code>curr = node(1).right</code> (2)
5	2	[2]	[1]	curr 不为空，压栈, <code>curr = curr.left</code> (3)
6	3	[2, 3]	[1]	curr 不为空，压栈, <code>curr = curr.left</code> (<code>null</code>)
7	null	[2, 3]	[1]	curr 为空，弹栈 (3)，访问 3
8	null	[2]	[1, 3]	<code>curr = node(3).right</code> (<code>null</code>)
9	null	[2]	[1, 3]	curr 为空，弹栈 (2)，访问 2
10	null	[]	[1, 3, 2]	<code>curr = node(2).right</code> (<code>null</code>)
11	null	[]	[1, 3, 2]	curr 为空，stack 为空，循环结束

方法三: Morris 遍历 (空间复杂度 $O(1)$ 的迭代) Morris 遍历是一种巧妙的迭代方法，它可以在 $O(1)$ 的额外空间复杂度下完成二叉树的中序遍历（以及前序、后序）。其核心思想是利用树中大量空闲的右指针（叶子节点的右指针，以及某些内部节点的右指针）来临时指向其中序遍历下的后继节点，从而在遍历过程中找到回溯到父节点的路径，而无需使用栈。

基本步骤:

1. 初始化当前节点 `curr = root`。
2. 当 `curr` 不为空时, 进行循环:
 - 如果 `curr` 没有左子节点:
 - 访问 `curr` 节点 (将其值加入 `result`)。
 - 将 `curr` 移动到其右子节点 (`curr = curr.right`)。
 - 如果 `curr` 有左子节点:
 - 找到 `curr` 左子树中最右侧的节点 (即 `curr` 在中序遍历中的前驱节点), 记为 `predecessor`。
 - * `predecessor = curr.left`
 - * `while (predecessor.right != null && predecessor.right != curr):`
 - `predecessor = predecessor.right`
 - 检查 `predecessor` 的右指针:
 - * 如果 `predecessor.right` 为空:
 - 说明是第一次访问到 `curr` (在处理其左子树之前)。
 - 将 `predecessor.right` 指向 `curr` (建立临时回溯链接)。
 - 将 `curr` 移动到其左子节点 (`curr = curr.left`), 继续处理左子树。
 - * 如果 `predecessor.right` 指向 `curr`:
 - 说明 `curr` 的左子树已经遍历完毕 (通过之前建立的临时链接回溯到了 `curr`)。
 - 访问 `curr` 节点 (将其值加入 `result`)。
 - 断开临时链接: 将 `predecessor.right` 恢复为空 (`predecessor.right = null`)。
 - 将 `curr` 移动到其右子节点 (`curr = curr.right`), 开始处理右子树。
3. 循环结束后, `result` 即为中序遍历结果。

优点: 空间复杂度为 $O(1)$ 。**缺点:** 实现相对复杂, 并且会在遍历过程中临时修改树的结构 (虽然最后会恢复)。

3. 代码实现 (Python)

```
# Definition for a binary tree node.
# class TreeNode:
#     def __init__(self, val=0, left=None, right=None):
#         self.val = val
#         self.left = left
#         self.right = right

from typing import List, Optional

class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
```

```

        self.left = left
        self.right = right

class Solution:
    # 方法一：递归
    def inorderTraversalRecursive(self, root: Optional[TreeNode]) -> List[int]:
        """
        使用递归实现中序遍历
        """
        result = []
        def traverse(node):
            # 递归终止条件：如果节点为空，则返回
            if not node:
                return
            # 递归遍历左子树
            traverse(node.left)
            # 访问当前节点（将值添加到结果列表）
            result.append(node.val)
            # 递归遍历右子树
            traverse(node.right)
        # 从根节点开始递归
        traverse(root)
        # 返回结果列表
        return result

    # 方法二：迭代（使用栈）
    def inorderTraversalIterative(self, root: Optional[TreeNode]) -> List[int]:
        """
        使用栈模拟递归实现中序遍历
        """
        result = [] # 存储结果
        stack = [] # 辅助栈，存储待访问的父节点
        curr = root # 当前节点指针
        # 当 curr 不为空（还有节点需要处理）或栈不为空（还有父节点待回溯）时，继续循环
        while curr or stack:
            # 阶段一：一路向左，将路径上的节点压入栈
            # 直到 curr 为空，表示到达当前子树的最左下角
            while curr:
                stack.append(curr) # 将当前节点压栈，以便后续回溯
                curr = curr.left # 移动到左子节点

            # 阶段二：curr 为空，说明左子树处理完毕（或无左子树）
            # 从栈顶弹出一个节点进行访问，这个节点是当前应该访问的“根”节点
            curr = stack.pop() # 弹出栈顶节点（最近的未访问父节点）
            result.append(curr.val) # 访问该节点（加入结果）

```

```

        # 转向右子树, 准备处理右子树
        # 将 curr 指向弹出节点的右子节点
        # 如果右子节点存在, 下一轮外层循环会对其执行阶段一 (一路向左)
        # 如果右子节点为空, 下一轮外层循环 curr 为空, 会继续执行阶段二 (弹出栈中下一个节点)
        curr = curr.right

    return result # 返回最终结果

# 方法三: Morris 遍历 (空间 O(1))
def inorderTraversalMorris(self, root: Optional[TreeNode]) -> List[int]:
    """
    使用 Morris 遍历实现中序遍历, 空间复杂度 O(1)
    """
    result = []
    curr = root # 当前节点指针
    while curr: # 当前节点不为空时循环
        if not curr.left:
            # 1. 如果当前节点没有左子节点
            #     - 直接访问该节点
            #     - 移动到右子节点
            result.append(curr.val)
            curr = curr.right
        else:
            # 2. 如果当前节点有左子节点
            #     找到当前节点在中序遍历下的前驱节点 (predecessor)
            #     前驱节点是其左子树中最右侧的节点
            predecessor = curr.left
            # 循环查找前驱节点, 条件是前驱节点的右指针不为空, 并且不指向当前节点 (避免死循环)
            while predecessor.right and predecessor.right != curr:
                predecessor = predecessor.right

            # 检查前驱节点的右指针
            if not predecessor.right:
                # 2.1 如果前驱节点的右指针为空
                #     - 说明是第一次访问到 curr (在处理其左子树之前)
                #     - 建立临时链接: 将前驱节点的右指针指向当前节点 curr
                predecessor.right = curr
                #     - 将当前节点移动到我左子节点, 继续处理左子树
                curr = curr.left
            else: # predecessor.right == curr
                # 2.2 如果前驱节点的右指针指向当前节点 curr
                #     - 说明 curr 的左子树已经遍历完毕 (通过之前建立的临时链接回溯到了 curr)
                #     - 断开临时链接, 恢复树的结构
                predecessor.right = None
                #     - 访问当前节点 curr (中序遍历访问根节点的时机)
                result.append(curr.val)

```

```

        # - 将当前节点移动到其右子节点，开始处理右子树
        curr = curr.right
    return result # 返回最终结果

# 最终提交可以选择其中一种方法
def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
    # return self.inorderTraversalRecursive(root) # 递归方法
    return self.inorderTraversalIterative(root) # 迭代（栈）方法
    # return self.inorderTraversalMorris(root) # Morris 遍历方法（空间 O(1)）

```

4. 复杂度分析

- **递归方法：**
 - 时间复杂度： $O(N)$ ，其中 N 是树中的节点数。每个节点被访问一次。
 - 空间复杂度： $O(H)$ ，其中 H 是树的高度。空间消耗主要来自递归调用栈。最坏情况下（链状树）为 $O(N)$ ，平均情况下（平衡树）为 $O(\log N)$ 。
- **迭代方法：**
 - 时间复杂度： $O(N)$ 。每个节点被压入和弹出栈各一次。
 - 空间复杂度： $O(H)$ 。栈中最多存储 H 个节点。最坏情况下为 $O(N)$ ，平均情况下为 $O(\log N)$ 。
- **Morris 遍历方法：**
 - 时间复杂度： $O(N)$ 。虽然看起来有嵌套循环，但仔细分析可知，每个节点最多被访问两次（一次是 `curr` 指向它，另一次可能是在查找前驱节点时通过 `predecessor.right` 访问到），每条边最多被遍历两次（一次向下，一次可能通过临时链接向上）。因此总时间复杂度是线性的。
 - 空间复杂度： $O(1)$ 。只使用了常数个额外指针（`curr`, `predecessor`）。

5. 如何在面试中讲解（更详细）

1. **明确定义与目标：**
 - “首先，明确中序遍历的定义：对于任何节点，我们先访问其左子树，然后访问该节点本身，最后访问其右子树。目标是返回一个包含按中序遍历顺序排列的节点值的列表。”
2. **递归方法阐述：**
 - “最自然的方法是递归。我们可以定义一个辅助函数 `traverse(node)`。”
 - “讲解递归三部曲：**终止条件**（`node` 为空时返回），**递归左子树**（`traverse(node.left)`），**处理当前节点**（将 `node.val` 加入结果列表），**递归右子树**（`traverse(node.right)`）。”
 - “强调访问当前节点的时机是在左子树递归调用返回之后，右子树递归调用之前。”
 - “分析复杂度：时间复杂度是 $O(N)$ 因为每个节点访问一次。空间复杂度是 $O(H)$ ， H 是树的高度，用于存储递归调用栈帧。最好情况 $O(\log N)$ （平衡树），最坏情况 $O(N)$ （链状树）。”
3. **迭代方法阐述（重点）：**
 - “递归虽然简洁，但可能因调用栈过深导致栈溢出。我们可以使用迭代方法，通过显式使用栈来模拟递归过程。”
 - “解释核心数据结构：一个栈 `stack` 用于存储待访问的节点，一个指针 `curr` 用于追踪当前节点。”

- “详细描述迭代过程（结合上面的迭代过程示例讲解）：”
 - “外层循环条件是 `curr` 不为空或 `stack` 不为空。”
 - “**步骤 1（深入左子树）**：当 `curr` 不为空时，持续将其压入栈，并移动到左子节点，直到 `curr` 为空。这一步是在模拟递归调用链，找到最左边的节点。”
 - “**步骤 2（访问与转向右）**：当 `curr` 为空时，意味着左子树已处理完（或无左子树）。此时从栈顶弹出一个节点 `node`，这就是当前应该访问的节点。将其值加入结果列表。”
 - “然后，将 `curr` 指向弹出节点 `node` 的右子节点，准备处理右子树。如果右子节点存在，下一轮循环会重复步骤 1；如果右子节点为空，下一轮循环会继续弹出栈顶元素（即当前节点的父节点）。”
 - “可以通过一个简单的例子（如 `[1, null, 2, 3]`）手动画图或口头模拟栈和 `curr` 的变化过程，展示节点是如何按中序顺序被访问的。”
 - “分析复杂度：时间复杂度仍为 $O(N)$ ，因为每个节点入栈出栈各一次。空间复杂度也是 $O(H)$ ，因为栈的大小最多是树的高度。”
4. **对比与选择：**
- “递归代码更简洁易懂，但有栈溢出风险。迭代代码稍复杂，但空间效率更可控。”
 - “根据具体场景（如树的深度限制、性能要求、是否允许修改树结构）选择合适的方法。”
 - “可以补充提一下 Morris 遍历作为 $O(1)$ 空间复杂度的进阶解法，简述其利用空闲右指针建立临时链接的思想，但指出其实现复杂且会临时修改树结构。”
5. **边界条件与鲁棒性（强调）：**
- “强调代码需要处理**空树**（`root` 为 `null`）的情况，此时应返回空列表。”
 - “考虑**单节点树**的情况。”
 - “考虑**只有左子树**或**只有右子树**的链状树情况，确保逻辑正确。”
 - “对于迭代法，要确保循环条件（`curr` 或 `stack` 不为空）和内部逻辑能正确处理所有情况，不会遗漏节点或死循环。”
6. **代码展示与风格：**
- “展示清晰、命名规范、有适当注释的代码。在写迭代法时，注释清楚两个阶段的逻辑有助于理解。”